

MASTER 2 – IASD/GL
HAI912I DÉVELOPPEMENT MOBILE

PlantNanny

Application Flutter interagissant avec un ESP32



Auteurs :

HUREL Jérémy – N° 21907809
MACKOW Anaïs – N° 22016204

Janvier 2026

Table des matières

1	Introduction	3
2	Démarche de Conception	4
2.1	Architecture Logicielle	4
2.1.1	Architecture Globale du Système	4
2.1.2	Architecture de l'Application Flutter	5
2.1.3	Pattern Repository	6
2.2	Diagramme de Classes UML	6
2.2.1	Modèles de Données	6
2.2.2	Couche Repository	8
2.2.3	Couche Présentation	8
2.3	Diagrammes de Séquence	9
2.3.1	Authentification Utilisateur	9
2.3.2	Récupération des Lectures	10
2.3.3	Envoi d'une Commande d'Arrosage	10
2.4	Workflow et Navigation	11
2.4.1	Flux de Navigation	11
2.4.2	Cycle de Vie d'une Commande	13
2.5	Gestion des États avec Riverpod	14
3	Ergonomie et Expérience Utilisateur	15
3.1	Décision de Design	15
3.2	Identité Visuelle	15
3.3	Composants d'Interface	16
3.3.1	Carte de Plante	16
3.3.2	Historique de données et graphique	17
3.3.3	Contrôles d'Arrosage	18
3.4	Animations et Micro-interactions	19
3.4.1	Principes d'Animation	19
3.4.2	Animations Implicites	19
3.4.3	Transitions entre Écrans	20
3.5	Design Responsive	20
4	Intégration avec l'API RESTful	21
4.1	Spécification OpenAPI	21
4.1.1	Génération automatique du client	21
4.2	Architecture d'Intégration	22
4.3	Couche Repository	22
4.3.1	DevicesRepository	22

4.3.2	ReadingsRepository	23
4.3.3	CommandsRepository	23
4.4	Authentication	23
4.4.1	Intégration Firebase Authentication	23
4.4.2	Gestion de l'État d'Authentification	23
4.4.3	Transmission du Token	24
4.5	Gestion des Erreurs	24
4.6	Stratégie de Test	24
4.6.1	Repositories Fake	24
4.6.2	Tests d'Intégration	25
4.6.3	Tests de Bout en Bout	25
5	Implémentation des Fonctionnalités	26
5.1	Affichage des informations et données des capteurs	26
5.1.1	Dashboard principal	26
5.1.2	Indicateurs Visuels	27
5.2	Contrôle de la LED	27
5.2.1	Interface de Contrôle	27
5.3	Interface de Statistiques	28
5.3.1	Vue d'Ensemble	28
5.4	Contrôle de l'Arrosage (Pompe)	29
5.4.1	Page d'Arrosage	29
5.4.2	Déclenchement de l'Arrosage	29
5.4.3	Configuration de la Durée	29
5.5	Provisionnement Bluetooth	29
5.5.1	Principe du Provisionnement	29
5.5.2	Découverte des Capteurs	30
5.5.3	Envoi des Identifiants WiFi	30
5.5.4	Sécurité du Provisionnement	30
5.6	Persistance des Données	31
5.6.1	Architecture de Stockage	31
5.6.2	Mode Hors Ligne	31
6	Défis Rencontrés et Solutions Adoptées	32
7	Conclusion	35

Chapitre 1

Introduction

Dans le cadre de notre Master 2 en Informatique, nous avons été amenés à développer une application mobile Flutter nommée **PlantNanny**. Ce projet s'inscrit dans la continuité du projet ESP32 mené avec M. Mancheron, au cours duquel nous avons réalisé un montage sur TT-Go intégrant différents capteurs, ainsi qu'une API permettant l'envoi et l'exploitation des données collectées.

L'objectif principal est de concevoir une application intuitive permettant d'interagir avec les capteurs connectés dédiés à la surveillance et à l'arrosage des plantes. L'application sert de pont entre l'utilisateur et le matériel, offrant une visualisation des données environnementales et un contrôle des actionneurs.

Les objectifs définis pour ce projet sont les suivants :

1. **Affichage des données des capteurs** : Visualisation des mesures de température, humidité et luminosité sous différents formats (texte, graphiques, tableaux de bord).
2. **Contrôle des actionneurs** : Gestion de la LED et de la pompe d'arrosage via des commandes envoyées à l'ESP32.
3. **Interface de réglage des seuils** : Envoi d'un message d'alerte lorsque les seuils définis sont atteints, afin d'indiquer qu'un arrosage est nécessaire.
4. **Interface de statistiques** : Visualisation de l'historique et des tendances pour la surveillance et le débogage.
5. **Persistance des données** : Stockage des données dans Firebase et PostgreSQL pour les données utilisatuer.

Dans la suite de ce rapport, nous allons détailler notre démarche dans la conception de l'application, tant sur le plan technique que sur les aspects d'ergonomie et d'expérience utilisateur. Nous expliquerons comment nous avons intégré l'API RESTful ainsi que les fonctionnalités implémentées. Enfin, nous aborderons les défis rencontrés et les solutions qui en ont découlé.

Architecture Globale et Technologies utilisées

Chapitre 2

Démarche de Conception

2.1 Architecture Logicielle

2.1.1 Architecture Globale du Système

L'application PlantNanny s'inscrit dans une architecture distribuée de type client-serveur, conçue pour permettre la surveillance et le contrôle de capteurs ESP32 dédiés à l'entretien des plantes. Cette architecture comprend quatre composants principaux qui communiquent entre eux de manière synchrone et asynchrone.

Le **capteur ESP32 TTGO T-Display** constitue le point d'entrée des données. Équipé de capteurs de température, d'humidité et de luminosité, il effectue des mesures périodiques et les transmet au serveur via le protocole MQTT. Il peut également recevoir des commandes (arrosage, lecture forcée) et les exécuter localement.

Le **serveur REST API**, en Python avec le framework Connexion, joue le rôle de hub central. Il expose une API RESTful conforme à la spécification OpenAPI 3.0.3, gère l'authentification des utilisateurs via Firebase, persiste les données dans une base de donnée PostgreSQL, et assure la communication bidirectionnelle avec les capteurs via un broker MQTT.

L'**application Flutter** que nous avons développée constitue l'interface utilisateur. Elle consomme l'API REST pour afficher les données et envoyer des commandes, tout en offrant une expérience utilisateur fluide grâce à une architecture réactive.

Enfin, **Firebase** fournit le service cloud pour l'authentification pour la gestion des identités utilisateur.

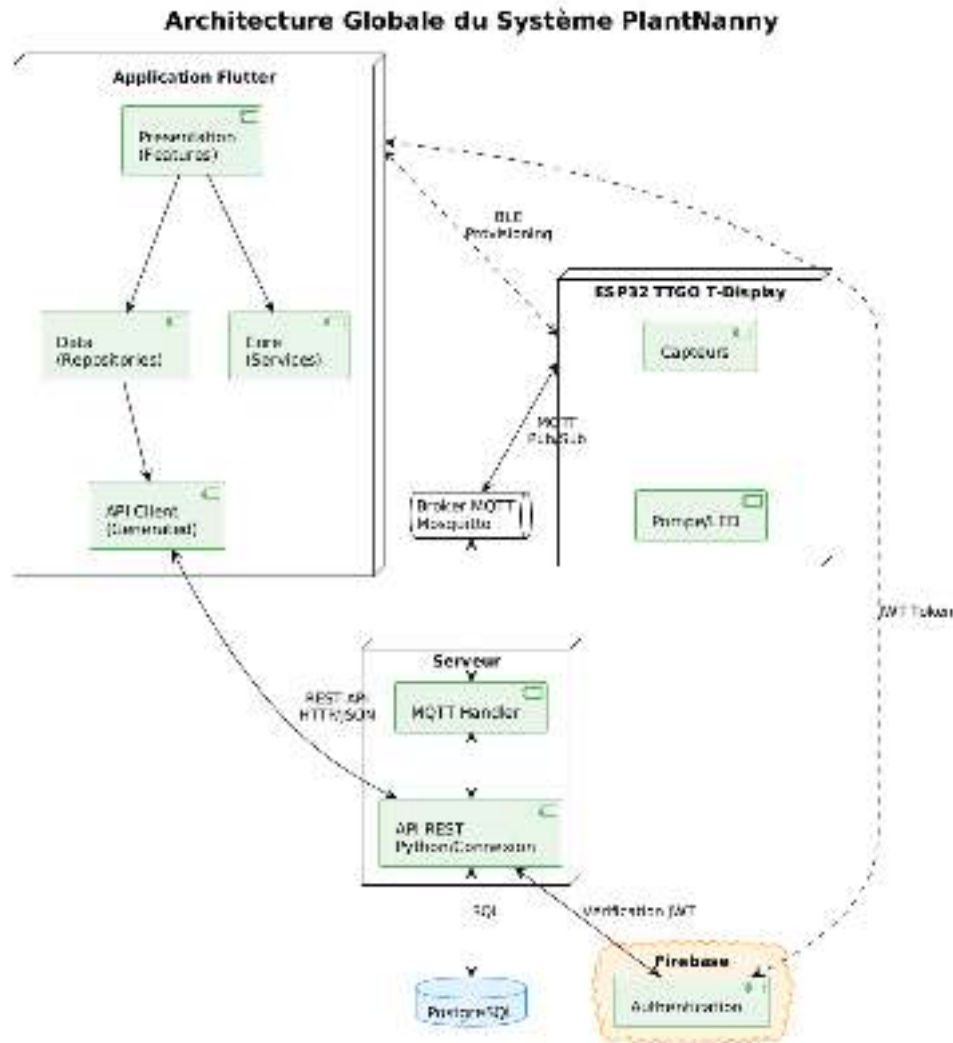


FIGURE 2.1 – Architecture globale du projet

2.1.2 Architecture de l'Application Flutter

L'application Flutter suit une architecture en couches inspirée de la **Clean Architecture**. Cette organisation permet de séparer clairement les responsabilités et de rendre le code plus testable et maintenable.

La **couche Présentation** (dossier `lib/features/`) contient l'ensemble des éléments visuels de l'application. Elle est organisée par fonctionnalité : authentification, tableau de bord, gestion des appareils, historique, etc. Chaque fonctionnalité regroupe ses pages (widgets de niveau écran), ses widgets réutilisables, et ses controllers qui gèrent l'état local.

La **couche Domaine** (dossier `lib/data/`) définit les règles métier de l'application. Elle contient les modèles de données (Device, Reading, Command) qui représentent les entités manipulées, ainsi que les repositories qui définissent les contrats d'accès aux données. Cette couche est indépendante de toute technologie externe.

La **couche Données** (dossier `lib/api/`) implémente concrètement l'accès aux sources de données. Elle contient le client API généré automatiquement à partir de la spécification OpenAPI, ainsi que les implémentations concrètes des repositories qui utilisent ce client.

La **couche Core** (dossier `lib/core/`) fournit les services transversaux utilisés par toutes les autres couches : configuration du serveur, service Bluetooth pour le provisionne-

ment WiFi, utilitaires divers.

TABLE 2.1 – Structure des couches de l’application

Couche	Dossier	Responsabilité
Présentation	lib/features/	Widgets, pages et controllers
Domaine	lib/data/	Repositories et modèles métier
Données	lib/api/	Communication avec l’API
Core	lib/core/	Services transversaux

2.1.3 Pattern Repository

Le pattern Repository constitue un élément central de notre architecture. Il agit comme une façade entre la logique métier et les sources de données, permettant d’abstraire complètement la manière dont les données sont obtenues ou persistées.

Concrètement, chaque type de données possède son propre repository. Le `DevicesRepository` gère la liste des capteurs de l’utilisateur, leur enregistrement et leur suppression. Le `ReadingsRepository` s’occupe de la récupération des mesures, qu’il s’agisse de la dernière lecture ou de l’historique complet. Le `CommandsRepository` permet d’envoyer des commandes aux capteurs.

L’un des avantages majeurs de ce pattern est la facilité de test qu’il procure. Pour chaque repository, nous avons créé une implémentation *fake* qui simule les réponses de l’API sans effectuer de véritables appels réseau. Ainsi, le `CommandsRepositoryFake` maintient une liste en mémoire des commandes envoyées, permettant de vérifier le comportement de l’application dans un environnement contrôlé.

Cette abstraction nous a également permis de développer l’interface utilisateur en parallèle du backend, en utilisant les implémentations fake jusqu’à ce que l’API soit disponible.

2.2 Diagramme de Classes UML

2.2.1 Modèles de Données

Le domaine métier de PlantNanny s’articule autour de quatre entités principales, dont les relations sont illustrées dans le diagramme de classes ci-dessous.

Diagramme de Classes - Modèles de Données PlantNanny

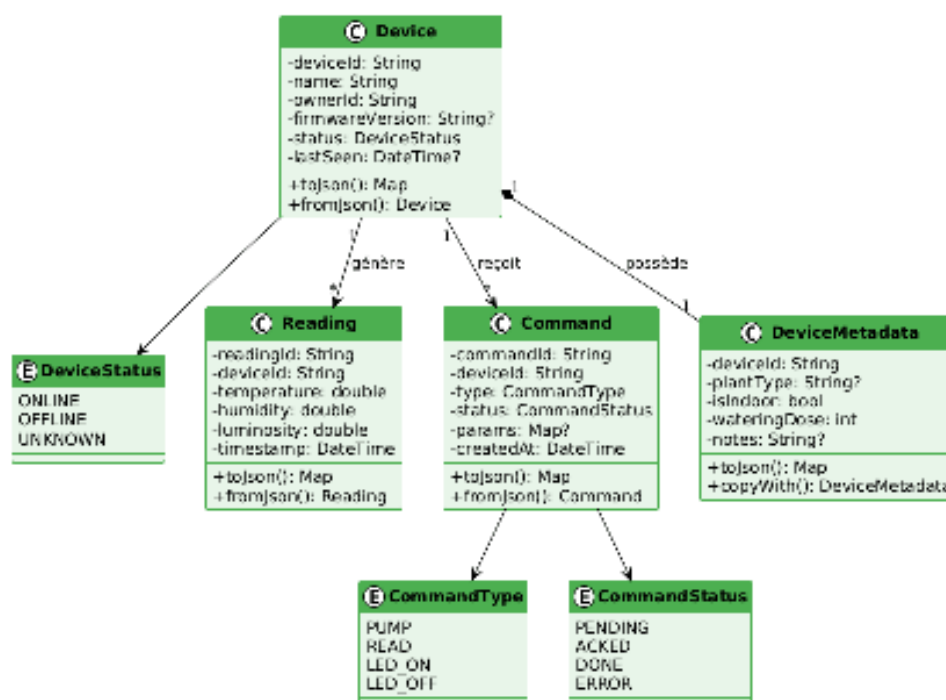


FIGURE 2.2 – Diagramme de classes des modèles de données

L'entité **Device** représente un capteur ESP32 enregistré dans le système. Elle contient l'identifiant unique du capteur, son nom personnalisé par l'utilisateur, l'identifiant du propriétaire, ainsi que des métadonnées techniques comme la version du firmware et la date de dernière connexion. Un Device est associé à un propriétaire unique et peut générer plusieurs lectures et recevoir plusieurs commandes.

L'entité **Reading** modélise une mesure effectuée par un capteur à un instant donné. Elle comprend les trois grandeurs mesurées qui sont la température en degrés Celsius, l'humidité relative en pourcentage, et la luminosité également en pourcentage. Chaque lecture est horodatée et rattachée à un capteur spécifique.

L'entité **Command** représente une action envoyée à un capteur. Elle possède un type (arrosage, lecture forcée, contrôle LED), un statut qui évolue au cours du temps (en attente, acquittée, terminée, erreur), et des paramètres optionnels comme la durée d'arrosage. Le cycle de vie d'une commande permet de suivre son exécution depuis l'envoi jusqu'à la confirmation par le capteur.

L'entité **DeviceMetadata** contient les informations complémentaires saisies par l'utilisateur (le type de plante surveillée, si elle est en intérieur ou extérieur, la dose d'arrosage de base, et des commentaires libres). Ces métadonnées permettent de personnaliser le comportement et l'affichage pour chaque plante.

2.2.2 Couche Repository

La couche Repository fait le lien entre les modèles de données et les sources externes. Le diagramme ci-dessous illustre l'organisation de cette couche et les relations entre ses composants.

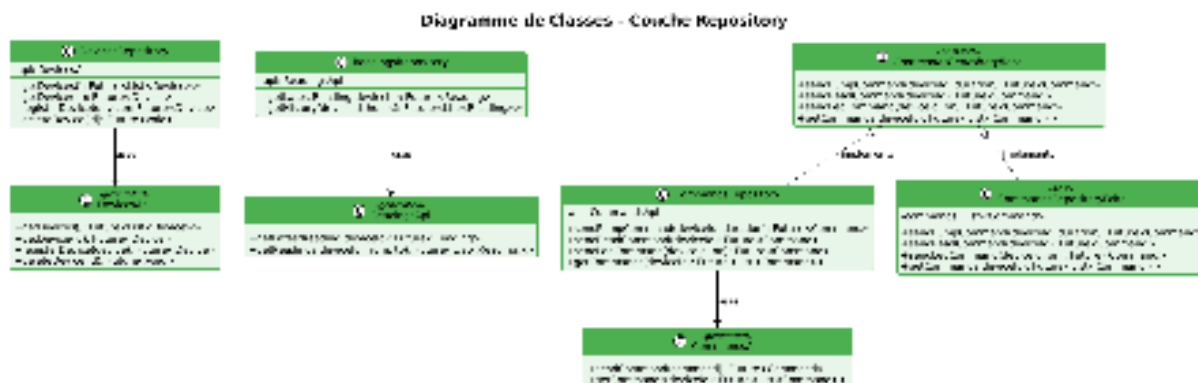


FIGURE 2.3 – Diagramme de classes de la couche Repository

Chaque repository concret dépend d'une classe API générée automatiquement. Le `DevicesRepository` utilise `DevicesApi` pour communiquer avec les endpoints relatifs aux capteurs. Cette dépendance est injectée via le constructeur, ce qui permet de la remplacer facilement par un mock lors des tests.

Pour le `CommandsRepository`, nous avons défini une interface abstraite `CommandsRepositoryBase` qui spécifie les opérations disponibles. Deux implémentations concrètes existent dont `CommandsRepository` qui effectue de vrais appels API et `CommandsRepositoryFake` qui simule les réponses. Cette structure permet de basculer entre les deux implémentations via l'injection de dépendances.

2.2.3 Couche Présentation

La couche présentation organise l'interface utilisateur autour de pages, de controllers et de widgets réutilisables.

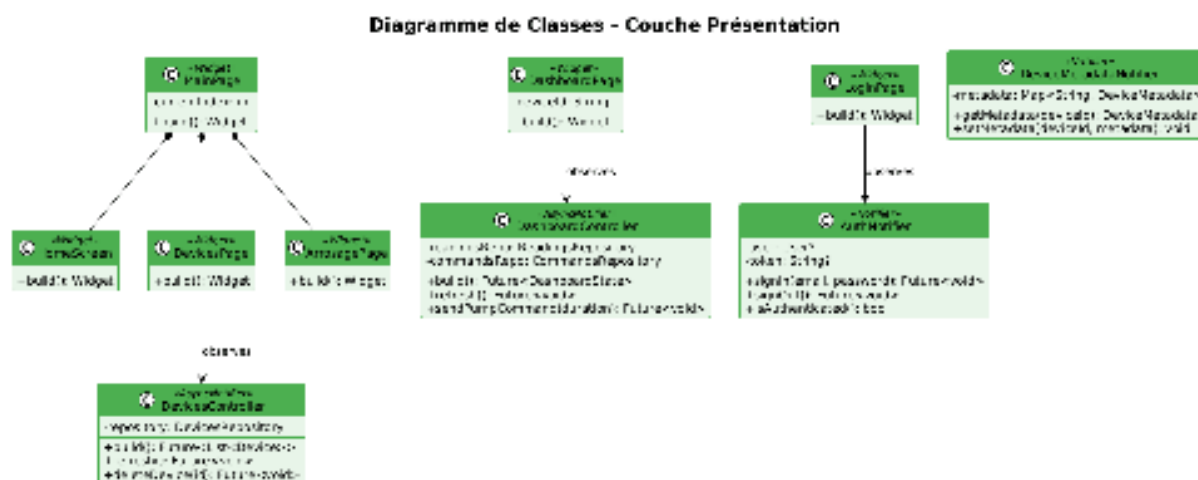


FIGURE 2.4 – Diagramme de classes de la couche Présentation

La **MainPage** constitue le point d'entrée après authentification. Elle gère la navigation principale via une barre de navigation inférieure et contient trois écrans : **HomeScreen** pour l'aperçu rapide, **DevicesPage** pour la liste des capteurs, et **ArrosagePage** pour le contrôle de l'arrosage.

Les controllers, comme **DevicesController** et **DashboardController**, encapsulent la logique de chargement et de rafraîchissement des données. Ils exposent un état asynchrone que les widgets observent pour se reconstruire automatiquement lors des mises à jour.

Les notifiers, comme **AuthNotifier** et **DeviceMetadataNotifier**, gèrent l'état global de l'application. L'**AuthNotifier** maintient l'état de connexion et le token JWT, tandis que le **DeviceMetadataNotifier** stocke les métadonnées des plantes en mémoire.

2.3 Diagrammes de Séquence

Les diagrammes de séquence permettent de visualiser les interactions entre les différents composants lors de l'exécution des cas d'usage principaux.

2.3.1 Authentification Utilisateur

Le processus d'authentification implique plusieurs acteurs : l'utilisateur, l'interface de connexion, le gestionnaire d'état, et le service Firebase.

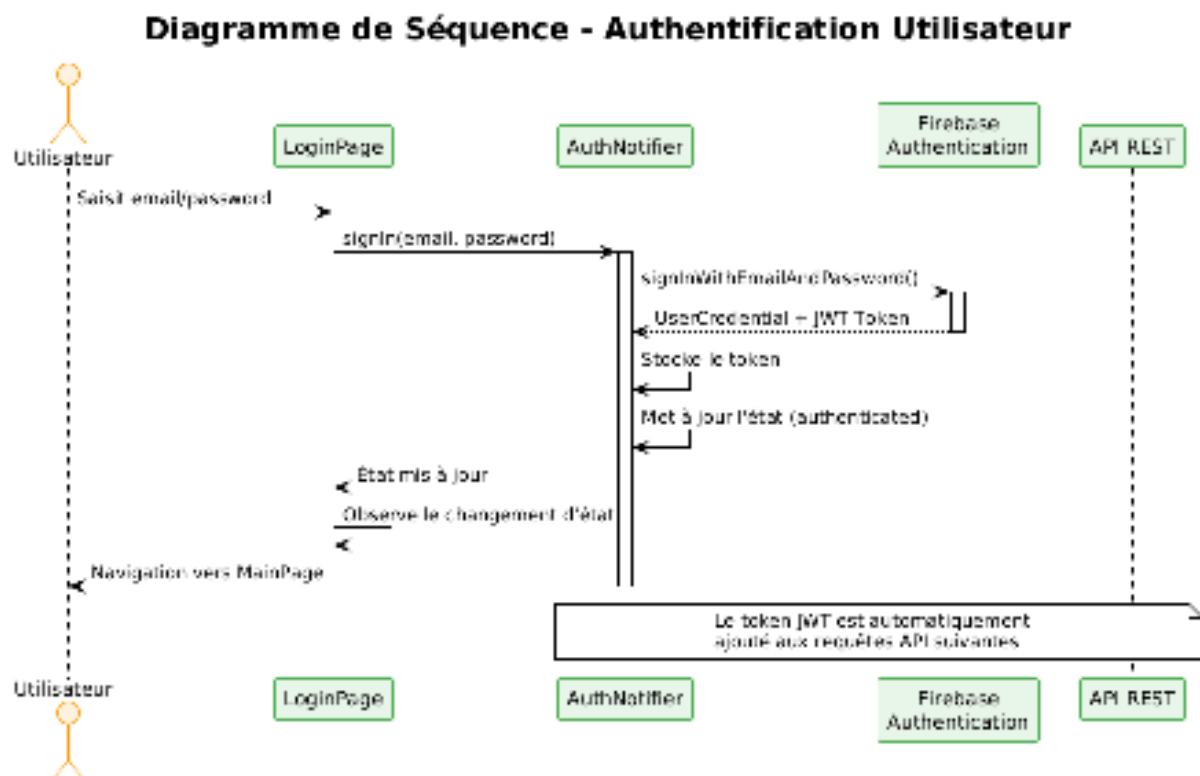


FIGURE 2.5 – Diagramme de séquence d'une authentification utilisateur

Lorsque l'utilisateur saisit ses identifiants et valide le formulaire, la page de connexion délègue l'opération à l'**AuthNotifier**. Celui-ci contacte **Firebase Authentication** avec l'email et le mot de passe. En cas de succès, **Firebase** retourne un objet utilisateur

contenant notamment un token JWT. L'AuthNotifieur stocke ce token et met à jour son état interne pour indiquer que l'utilisateur est connecté. La page de connexion qui observe cet état déclenche alors la navigation vers l'écran principal.

Ce token JWT est ensuite automatiquement ajouté à toutes les requêtes vers l'API REST, permettant au serveur de vérifier l'identité de l'utilisateur et de filtrer les données en conséquence.

2.3.2 Récupération des Lectures

La récupération des données de capteurs suit un flux plus simple, purement synchrone entre l'application et le serveur.

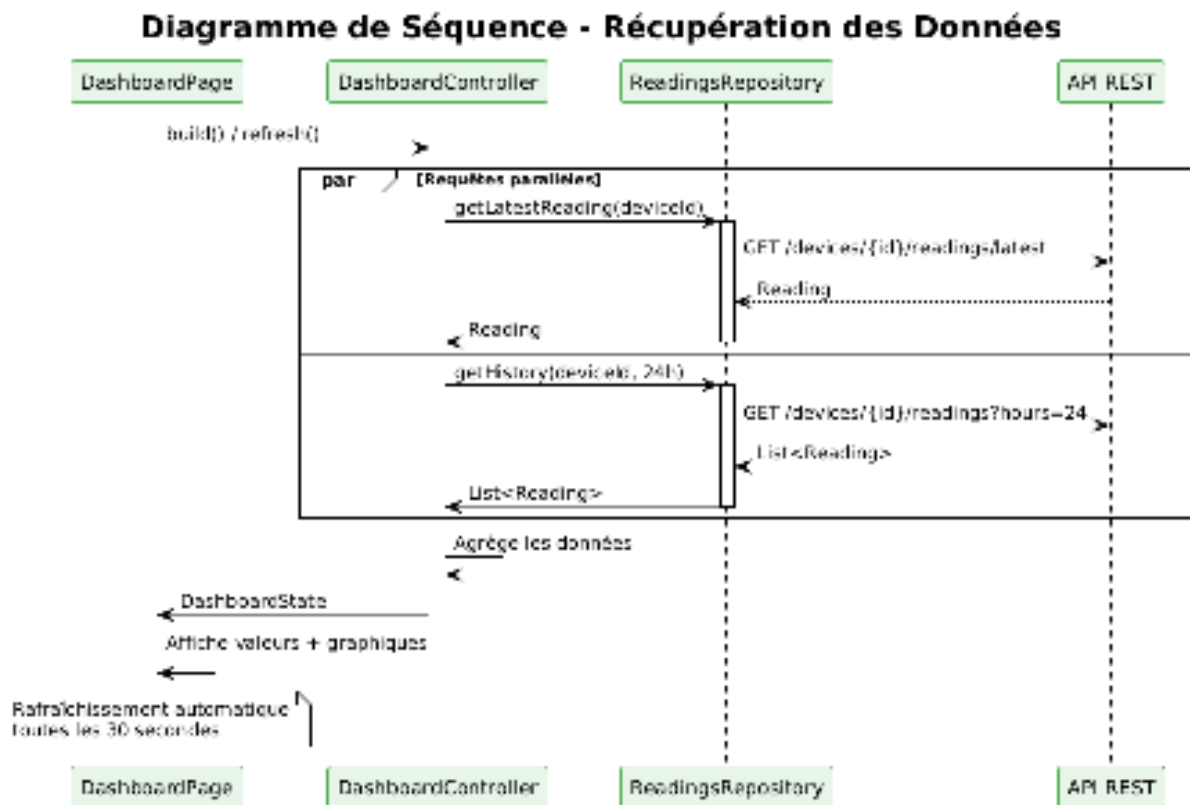


FIGURE 2.6 – Diagramme de séquence d'une récupération de données

Lorsque l'utilisateur accède au tableau de bord d'un capteur, le **DashboardController** effectue deux requêtes parallèles : une pour obtenir la dernière lecture, et une autre pour récupérer l'historique des 24 dernières heures. Ces données sont ensuite agrégées et exposées à l'interface qui les affiche sous forme de valeurs numériques et de graphiques.

Un mécanisme de rafraîchissement périodique (toutes les 30 secondes) permet de maintenir l'affichage à jour sans intervention de l'utilisateur.

2.3.3 Envoi d'une Commande d'Arrosage

L'envoi d'une commande d'arrosage illustre la communication entre l'application, le serveur REST et le capteur ESP32.

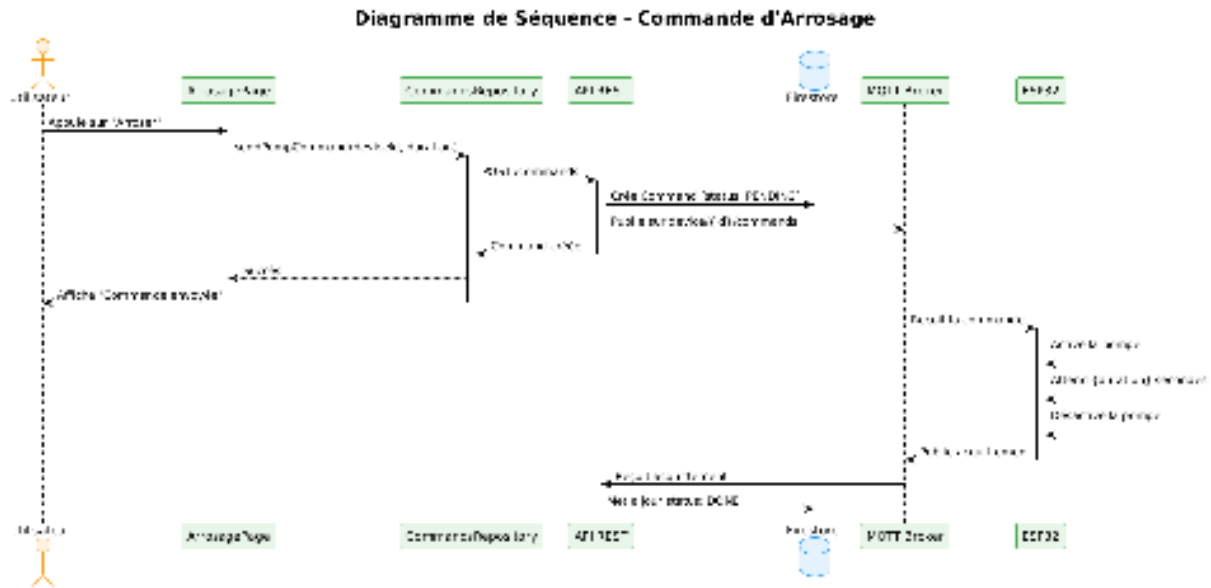


FIGURE 2.7 – Diagramme de séquence de lancement d’arrosage / allumage de LED

L’utilisateur déclenche l’arrosage depuis l’interface de l’application, en spécifiant éventuellement une durée. La page fait appel au `CommandsRepository`, qui transmet la requête à l’API. Le serveur crée alors une commande avec le statut *pending* et la persiste en base de données. Il publie ensuite un message MQTT à destination du capteur concerné.

Le capteur ESP32, abonné à son topic de commandes, reçoit l’instruction et active l’arrosage pour la durée spécifiée. Une fois l’opération terminée, il publie un message d’acquiescement, permettant au serveur de mettre à jour le statut de la commande à *done*.

Côté application, l’interface affiche immédiatement un retour visuel indiquant que la commande a bien été envoyée, sans attendre la fin de son exécution.

À noter que, par respect des consignes de sécurité et en raison de dysfonctionnements de la pompe, l’activation réelle de celle-ci a été remplacée par l’allumage d’une LED rouge pendant la durée théorique d’écoulement de l’eau.

2.4 Workflow et Navigation

2.4.1 Flux de Navigation

La navigation dans l’application suit une structure hiérarchique, illustrée dans le diagramme ci-dessous.

Au lancement, l'application vérifie l'état de la connexion de l'utilisateur. Si celui-ci n'est pas authentifié, il est redirigé vers le module d'authentification, lequel propose trois écrans : connexion, inscription et récupération de mot de passe.

Une fois authentifié, l'utilisateur accède à l'écran principal, structuré autour d'une navigation par onglets située en bas de l'interface. L'onglet Accueil offre une vue d'ensemble de toutes les plantes, accompagnées de leur état actuel. L'onglet Mes plantes liste les capteurs enregistrés et permet d'accéder aux informations détaillées de chacun. L'onglet Arrosage propose un contrôle rapide de l'arrosage pour l'ensemble des capteurs.

Depuis la liste des plantes, l'utilisateur peut naviguer vers le tableau de bord détaillé d'un capteur. Celui-ci affiche les mesures en temps réel, les graphiques historiques ainsi que des contrôles avancés.

Pour chaque capteur, l'utilisateur a également la possibilité de modifier son nom, d'associer une plante et d'ajouter un commentaire personnalisé.

2.4.2 Cycle de Vie d'une Commande

Les commandes envoyées aux capteurs suivent un cycle de vie bien défini, représenté par le diagramme d'états ci-dessous.

Diagramme d'États - Cycle de Vie d'une Commande

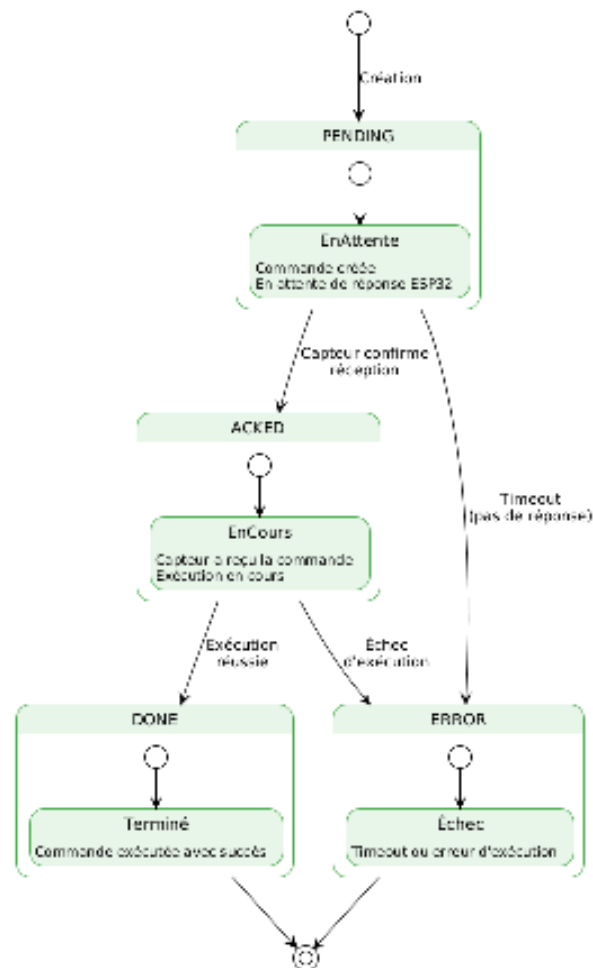


FIGURE 2.9 – Cycle de vie d'une commande

Une commande naît dans l'état **PENDING** lors de sa création par le serveur. Elle transite vers l'état **ACKED** lorsque le capteur confirme sa réception. Selon le résultat de l'exécution, elle atteint finalement l'état **DONE** en cas de succès ou **ERROR** en cas d'échec. Un timeout peut également faire passer une commande directement de **PENDING** à **ERROR** si le capteur ne répond pas dans le délai imparti.

Cette machine à états permet à l'interface de fournir un retour précis à l'utilisateur sur l'avancement de ses actions.

2.5 Gestion des États avec Riverpod

La gestion de l'état constitue un défi majeur dans le développement d'applications Flutter. Nous avons choisi **Riverpod** comme solution, pour plusieurs raisons.

Riverpod offre une **injection de dépendances type-safe** vérifiée à la compilation. Contrairement à d'autres solutions comme Provider, les erreurs de typage sont détectées avant l'exécution, réduisant les risques de bugs en production.

Le framework gère **automatiquement le cycle de vie** des providers. Un provider n'est instancié que lorsqu'il est effectivement utilisé, et il est automatiquement détruit lorsque plus aucun widget ne l'observe. Cela évite les fuites de mémoire et simplifie la gestion des ressources.

Les **AsyncNotifiers** facilitent la gestion des états asynchrones. Ils exposent un état de type **AsyncValue** qui peut représenter trois situations soit un chargement en cours ou données disponibles ou erreur. Les widgets peuvent réagir à chacun de ces états de manière déclarative.

Enfin, Riverpod permet de **surcharger facilement les providers** lors des tests. On peut remplacer un repository réel par sa version fake en une seule ligne, sans modifier le code de production.

Dans notre architecture, les controllers (comme **DevicesController**) sont des Async-Notifiers qui chargent les données au démarrage et exposent des méthodes pour rafraîchir ou modifier l'état. Les pages observent ces controllers et se reconstruisent automatiquement lors des changements d'état.

Chapitre 3

Ergonomie et Expérience Utilisateur

3.1 Décision de Design

Choix de Material Design 3 Pour l'interface de PlantNanny, nous avons opté pour **Material Design 3**. Ce langage de design développé par Google constitue la dernière évolution des principes Material et s'intègre nativement dans Flutter via le widget `MaterialApp`.

Ce choix repose sur plusieurs considérations stratégiques. Premièrement, Material Design 3 garantit une **cohérence visuelle cross-platform**. Le système de **thèmes dynamiques** permet d'adapter automatiquement la palette de couleurs à partir d'une couleur source unique. En définissant le vert nature comme couleur primaire, l'ensemble des teintes secondaires, des surfaces et des états interactifs sont générés de manière harmonieuse, tant en mode clair qu'en mode sombre.

Enfin, les composants Material respectent nativement les **guidelines d'accessibilité**, avec des contrastes suffisants, des zones tactiles dimensionnées correctement et un support intégré des lecteurs d'écran.

Enfin, l'écosystème Flutter dispose d'une documentation exhaustive pour Material Design, facilitant l'implémentation et la résolution des problèmes rencontrés.

Alternatives Considérées Nous avons évalué le style **Cupertino** (iOS natif) comme alternative. Si ce dernier offre une intégration parfaite sur iPhone, il présente plusieurs limitations pour notre projet comme une personnalisation restreinte, absence de support pour les thèmes dynamiques, et bibliothèque de composants plus limitée. De plus, ciblant également Android et le web, Material Design s'imposait comme le choix le plus polyvalent.

3.2 Identité Visuelle

Palette de Couleurs L'identité visuelle de PlantNanny s'articule autour d'une palette inspirée du monde végétal, renforçant intuitivement la thématique de l'application.

Le **vert principal** (`#4CAF50`) constitue la couleur dominante. Il évoque la nature, la croissance et la vitalité des plantes. Cette teinte est utilisée pour les actions principales, les éléments de navigation actifs et les indicateurs de succès.

Le **vert clair** (`#81C784`) sert de couleur secondaire pour les arrière-plans de cartes et les surfaces surélevées, créant une hiérarchie visuelle sans agressivité.

Le **bleu eau** (#2196F3) accompagne spécifiquement les indicateurs d’humidité et d’arrosage, établissant une association sémantique immédiate avec l’eau.

L’**orange** (#FF9800) signale les avertissements : seuils approchés, capteur déconnecté depuis peu, batterie faible.

Le **rouge** (#F44336) est réservé aux situations critique : erreurs de connexion, plante nécessitant un arrosage urgent, échec d’une commande. Son utilisation parcimonieuse lui conserve son impact.

Typographie La hiérarchie typographique structure l’information sur chaque écran. Les **titres principaux** (display/headline) utilisent une graisse élevée pour marquer les sections. Les **titres secondaires** accompagnent les cartes et les groupes de contenu. Le **corps de texte** privilégie la lisibilité avec une taille confortable de 14 à 16 points. Les **labels** plus petits annotent les graphiques et les valeurs numériques.

La police Poppins, géométrique et moderne, apporte une touche contemporaine tout en restant parfaitement lisible sur les petits écrans.

3.3 Composants d’Interface

3.3.1 Carte de Plante

Le composant `DevicePreviewCard` constitue l’élément visuel central de l’application. Cette carte synthétise l’état d’une plante en un coup d’œil et invite à l’interaction.

Visuellement, la carte présente le **nom de la plante** en titre, accompagné d’un indicateur coloré de statut (en ligne/hors ligne). Les **trois mesures principales** (température, humidité, luminosité) s’affichent avec leurs icônes respectives et un code couleur indiquant si les valeurs sont dans la plage optimale.

Un **badge d’alerte** apparaît dynamiquement lorsque la plante nécessite une attention particulière : arrosage recommandé, température trop basse, ou capteur déconnecté. Une subtile **animation de pulsation** attire le regard sur les cartes requérant une action.

Le tap sur la carte conduit au tableau de bord détaillé, tandis qu’un appui long révèle un menu contextuel avec les actions rapides (arroser, rafraîchir les données).



FIGURE 3.1 – Menu principal

3.3.2 Historique de données et graphique

La visualisation des données temporelles utilise un composant de graphique linéaire personnalisé. Plutôt que d'intégrer une bibliothèque externe lourde, nous avons développé un widget léger basé sur `CustomPainter`.

Ce graphique trace l'évolution d'une mesure (température, humidité ou luminosité) sur une période configurable (24 heures, 7 jours). La courbe utilise un lissage pour améliorer la lisibilité, et un dégradé sous la ligne renforce visuellement l'amplitude des variations.

L'interaction tactile permet de faire glisser un curseur sur la courbe pour afficher la valeur exacte à chaque instant, avec l'horodatage correspondant dans une infobulle.

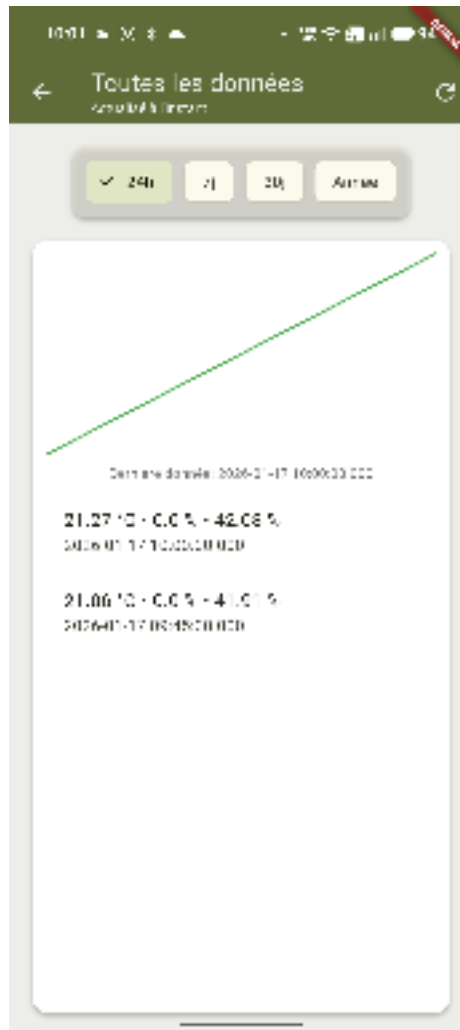


FIGURE 3.2 – Statistiques et Historique

3.3.3 Contrôles d'Arrosage

L'écran d'arrosage propose une interface épurée centrée sur l'action. Un **slider** permet de sélectionner la durée d'arrosage (de 1 à 30 secondes), avec un retour visuel immédiat de la quantité d'eau estimée.

Le **bouton d'arrosage**, proéminent et coloré en bleu, déclenche la commande. Durant l'envoi, il affiche un indicateur de progression, puis confirme le succès par un changement de couleur et une animation de validation.

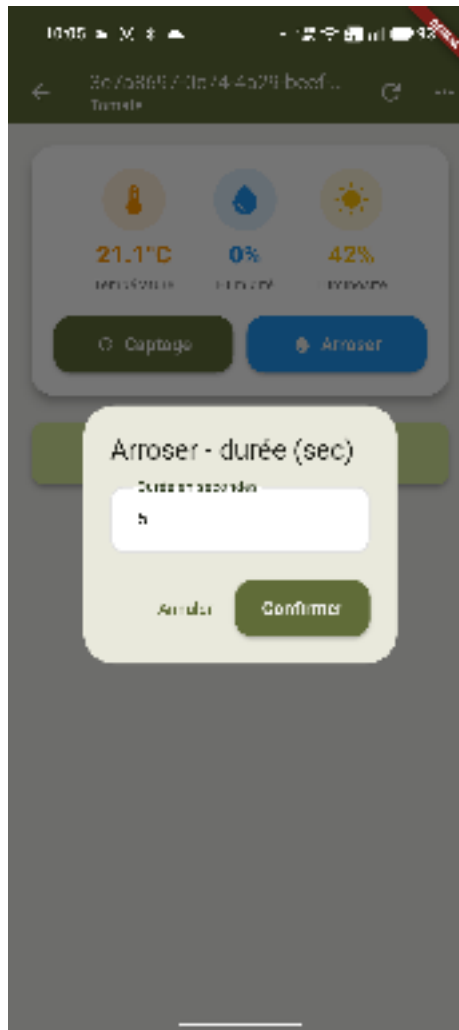


FIGURE 3.3 – Interface d’arrosage

3.4 Animations et Micro-interactions

3.4.1 Principes d’Animation

Les animations dans PlantNanny doivent être **fonctionnelles** (guider l’attention, indiquer un changement d’état), **subtiles** (ne pas distraire ni ralentir l’utilisateur) et **cohérentes** (mêmes durées et courbes d’accélération dans toute l’application).

La durée standard est de 300 millisecondes pour les transitions courtes, avec une courbe `easeInOut` qui démarre et termine en douceur.

3.4.2 Animations Implicites

Flutter propose des widgets d’animation implicite qui simplifient l’implémentation. Le composant `AnimatedContainer` permet de transitionner automatiquement entre deux états visuels (couleur, taille, bordures) sans gérer manuellement les contrôleurs d’animation.

Nous l’utilisons notamment pour les changements de sélection dans les listes, les variations de couleur selon l’état des mesures, et les ajustements de mise en page.

3.4.3 Transitions entre Écrans

Les transitions de navigation utilisent le mécanisme **Hero** de Flutter. Lorsque l'utilisateur sélectionne une carte de plante, celle-ci "vole" vers sa position dans l'écran de détail, créant une continuité visuelle qui aide à comprendre la relation entre les deux vues.

3.5 Design Responsive

Stratégie d'Adaptation L'application s'adapte à trois catégories de tailles d'écran grâce au widget **LayoutBuilder** qui permet de construire conditionnellement l'interface selon les dimensions disponibles.

Sur **smartphone** (largeur inférieure à 600 points), l'interface utilise une mise en page en colonne unique. La navigation principale s'effectue via la barre d'onglets en bas de l'écran. Les cartes de plantes occupent toute la largeur.

Sur **tablette portrait** (600 à 900 points), les cartes s'organisent en grille de deux colonnes. L'écran de détail peut afficher le graphique et les contrôles côte à côte.

Sur **tablette paysage et desktop** (plus de 900 points), une navigation latérale remplace les onglets inférieurs. La vue master-détail permet d'afficher la liste des plantes et le détail simultanément, évitant les allers-retours.

Grille Flexible La grille de cartes utilise un système flexible qui calcule le nombre de colonnes optimal selon la largeur disponible, garantissant que chaque carte conserve une taille lisible tout en exploitant au mieux l'espace.

Chapitre 4

Intégration avec l'API RESTful

4.1 Spécification OpenAPI

L'API RESTful PlantNanny constitue le point de communication central entre l'application mobile et le serveur backend. Elle est entièrement spécifiée au format **OpenAPI 3.0.3**, un standard industriel qui permet de décrire formellement les endpoints, les paramètres attendus, les formats de réponse et les codes d'erreur possibles.

Le fichier de spécification, situé dans `api/plant_nanny_api.yaml`, sert de contrat entre le frontend et le backend. Cette approche "API-first" présente nous a permis de documenter automatiquement l'API et de générer des clients dans différents langages en garantissant la cohérence entre la spécification et l'implémentation.

L'API suit les conventions REST suivantes :

- les ressources sont identifiées par des URLs
- les opérations utilisent les verbes HTTP appropriés (GET pour la lecture, POST pour la création)
- les réponses sont formatées en JSON

Méthode	Endpoint	Description
GET	<code>/v1/devices</code>	Liste des capteurs de l'utilisateur
POST	<code>/v1/devices/register</code>	Enregistrement d'un nouveau capteur
GET	<code>/v1/devices/{id}/readings</code>	Historique des mesures
GET	<code>/v1/devices/{id}/readings/last</code>	Dernière mesure
POST	<code>/v1/devices/{id}/commands</code>	Envoi d'une commande
GET	<code>/v1/devices/{id}/commands/pending</code>	Commandes en attente

TABLE 4.1 – Endpoints principaux de l'API

4.1.1 Génération automatique du client

Plutôt que d'écrire manuellement le code d'appel à l'API, nous avons utilisé le générateur **OpenAPI Generator** pour créer automatiquement un client Dart à partir de la spécification YAML. Un script dédié dans le dossier `Devtools/` exécute cette génération.

Le client généré comprend trois catégories d'éléments. Les **modèles de données** correspondent aux schémas définis dans la spécification : Device, Reading, Command, et leurs variantes. Les **classes API** regroupent les méthodes d'appel par domaine fonctionnel : DevicesApi, ReadingsApi, CommandsApi. Enfin, la **sérialisation JSON** est gérée automatiquement, convertissant les réponses JSON en objets Dart typés.

4.2 Architecture d'Intégration

Configuration Centralisée La configuration du client API est gérée de manière centralisée via le système de providers Riverpod. Un provider dédié crée et configure l'instance du client API, en définissant l'URL de base du serveur et en ajoutant automatiquement le token d'authentification à chaque requête.

Cette centralisation présente plusieurs avantages. Le changement d'environnement (développement, production) s'effectue en un seul point. Le token JWT est injecté automatiquement sans que chaque repository ait à s'en préoccuper. Les paramètres communs (timeout, headers par défaut) sont configurés une seule fois.

Gestion des Environnements L'application supporte plusieurs environnements de déploiement, chacun caractérisé par une URL de serveur différente. L'environnement de **développement** pointe vers un serveur local (localhost), facilitant le débogage et les tests rapides. L'environnement de **production** utilise l'URL du serveur déployé.

La configuration active est déterminée au lancement de l'application, permettant aux développeurs de basculer facilement entre les environnements sans modifier le code.

4.3 Couche Repository

Les repositories constituent la couche d'abstraction entre l'application et l'API. Ils encapsulent les appels au client généré et exposent des méthodes métier plus lisibles. Cette séparation permet de masquer les détails techniques de l'API (noms de méthodes générés, gestion des réponses nulles) derrière une interface claire et stable.

Chaque type de ressource dispose de son propre repository, respectant le principe de responsabilité unique.

4.3.1 DevicesRepository

Le **DevicesRepository** gère toutes les opérations relatives aux capteurs ESP32. La méthode `list()` récupère l'ensemble des capteurs appartenant à l'utilisateur authentifié. La méthode `register()` enregistre un nouveau capteur dans le système, en associant son code d'appairage à un nom personnalisé. La méthode `unregister()` permet de supprimer un capteur du compte utilisateur.

En cas d'erreur de l'API (réponse nulle, code d'erreur), le repository convertit ces situations en exceptions typées que les couches supérieures peuvent traiter de manière appropriée.

4.3.2 ReadingsRepository

Le **ReadingsRepository** s'occupe de la récupération des mesures effectuées par les capteurs. La méthode `last()` retourne la mesure la plus récente d'un capteur donné, utilisée pour l'affichage en temps réel sur le tableau de bord. Si aucune mesure n'existe encore, la méthode retourne une valeur nulle plutôt qu'une erreur.

La méthode `history()` récupère l'historique des mesures sur une période donnée, avec possibilité de limiter le nombre de résultats et de filtrer par date. Ces données alimentent les graphiques d'évolution.

La méthode `aggregate()` retourne des données agrégées (moyennes horaires, journalières, hebdomadaires), optimisées pour l'affichage de tendances sur de longues périodes sans surcharger le réseau avec des milliers de points individuels.

4.3.3 CommandsRepository

Le **CommandsRepository** permet d'envoyer des commandes aux capteurs et de suivre leur exécution. La méthode `pump()` envoie une commande d'arrosage avec une durée spécifiée en millisecondes. La méthode `forceReading()` demande au capteur d'effectuer une mesure immédiate plutôt que d'attendre le prochain cycle programmé.

La méthode `pending()` liste les commandes en attente d'exécution pour un capteur donné, permettant à l'interface d'afficher l'état des commandes récemment envoyées.

Ce repository implémente une interface abstraite **CommandsRepositoryBase**, facilitant la substitution par une implémentation fake lors des tests.

4.4 Authentification

4.4.1 Intégration Firebase Authentication

L'authentification des utilisateurs repose sur **Firebase Authentication**, le service cloud de **Google** qui prend en charge la création de comptes, la connexion, la réinitialisation de mot de passe et la gestion des sessions.

Lorsqu'un utilisateur se connecte avec son email et son mot de passe, Firebase vérifie les identifiants et retourne un **token JWT** (JSON Web Token). Ce token, signé cryptographiquement par Firebase, contient l'identifiant unique de l'utilisateur et une date d'expiration.

4.4.2 Gestion de l'État d'Authentification

Un notifieur Riverpod dédié, l'**AuthNotifier**, gère l'état d'authentification de l'application. Il écoute les changements d'état de Firebase Authentication et met à jour l'état local en conséquence.

Au démarrage de l'application, Firebase restaure automatiquement la session si l'utilisateur était précédemment connecté. L'**AuthNotifier** détecte cette restauration et initialise l'état comme authentifié, évitant à l'utilisateur de se reconnecter à chaque lancement.

Le token JWT est automatiquement rafraîchi par Firebase avant son expiration. L'**AuthNotifier** récupère le nouveau token et met à jour la configuration du client API, assurant une continuité de service transparente.

4.4.3 Transmission du Token

Le token JWT est ajouté automatiquement à chaque requête vers l'API REST sous forme d'en-tête `HTTP Authorization: Bearer <token>`. Le serveur vérifie la validité du token auprès de Firebase et extrait l'identifiant utilisateur pour filtrer les données retournées.

Cette architecture permet au serveur de ne jamais stocker de mots de passe et de déléguer entièrement la gestion des identités à Firebase, réduisant la surface d'attaque et simplifiant la maintenance.

4.5 Gestion des Erreurs

Hiérarchie des Exceptions Les erreurs pouvant survenir lors des appels API sont classifiées en plusieurs catégories, chacune représentée par une classe d'exception dédiée.

Les **NetworkException** signalent les problèmes de connectivité (absence de connexion internet, timeout de la requête, serveur injoignable). Ces erreurs sont généralement temporaires et l'utilisateur peut réessayer.

Les **AuthenticationException** indiquent un problème d'authentification par un token expiré, un token invalide ou accès refusé. Ces erreurs nécessitent une reconnexion de l'utilisateur.

Les **ApiException** englobent les erreurs métier retournées par le serveur (ressource non trouvée, validation échouée, conflit de données). Le code HTTP et le message d'erreur sont préservés pour permettre un traitement approprié.

Traitement Centralisé Un intercepteur centralisé enveloppe tous les appels API et transforme les exceptions techniques en exceptions métier compréhensibles. Les erreurs de socket deviennent des **NetworkException** avec un message explicite. Les erreurs 401 déclenchent automatiquement une **AuthenticationException**. Les autres erreurs HTTP sont converties en **ApiException** avec leur code et message.

4.6 Stratégie de Test

4.6.1 Repositories Fake

Pour tester l'application indépendamment du serveur, chaque repository dispose d'une implémentation "fake" qui simule les réponses de l'API en mémoire. Le **DevicesRepositoryFake** maintient une liste de capteurs fictifs, le **ReadingsRepositoryFake** génère des mesures aléatoires réalistes, et le **CommandsRepositoryFake** simule l'exécution des commandes avec des délais appropriés.

Ces implémentations fake permettent de développer et tester l'interface utilisateur sans dépendre de la disponibilité du serveur. Elles sont également utilisées pour les tests unitaires et d'intégration de l'application.

4.6.2 Tests d'Intégration

Les tests d'intégration vérifient que les repositories interagissent correctement avec le client API généré. Ils utilisent des mocks du client pour simuler différents scénarios :

- réponses réussies
- erreurs réseau
- erreurs d'authentification

Ces tests valident notamment que les conversions de données sont correctes, que les erreurs sont propagées de manière appropriée, et que les cas limites (listes vides, valeurs nulles) sont gérés sans crash.

4.6.3 Tests de Bout en Bout

Des tests de bout en bout, exécutés contre un serveur de test, vérifient l'intégration complète de la chaîne : authentification, récupération des données, envoi de commandes. Ces tests détectent les incompatibilités entre le client généré et le serveur réel, notamment après une mise à jour de l'API.

Chapitre 5

Implémentation des Fonctionnalités

5.1 Affichage des informations et données des capteurs

5.1.1 Dashboard principal

Le tableau de bord constitue l'écran principal de l'application, offrant à l'utilisateur une vue synthétique de l'état de ses plantes. Il agrège les informations issues de l'ensemble des capteurs enregistrés et les présente de manière claire, structurée et hiérarchisée.

Informations sur un capteur précis

L'affichage textuel présente les données brutes de manière lisible et compréhensible. Chaque mesure est accompagnée de son unité appropriée. Ainsi la température est exprimée en degrés Celsius avec une décimale de précision, tandis que l'humidité et la luminosité sont affichées en pourcentage, arrondies à l'unité. L'horodatage de chaque mesure est indiqué sous forme relative (par exemple « il y a 5 minutes ») ou absolue selon l'ancienneté des données, permettant à l'utilisateur d'évaluer rapidement leur fraîcheur.

En complément des données mesurées, l'interface offre des fonctionnalités de personnalisation pour chaque capteur. L'utilisateur peut modifier le nom du capteur afin de l'identifier plus facilement, ainsi que le type de plante associée. La sélection de la plante s'effectue via un menu déroulant intégrant une barre de recherche : si la plante recherchée n'existe pas encore dans la base de données, elle est automatiquement enregistrée lors de la validation.

Il est également possible d'ajouter des commentaires personnalisés pour chaque capteur, permettant à l'utilisateur de consigner des observations ou des informations spécifiques liées à la plante ou à son environnement.

Ce format d'affichage et de configuration convient aux utilisateurs souhaitant disposer d'informations précises, personnalisables et non ambiguës sur l'état de leurs plantes.

Graphiques et Charts

Les graphiques d'évolution constituent le mode de visualisation le plus informatif. L'application utilise des courbes linéaires pour représenter l'évolution temporelle de chaque paramètre : température, humidité du sol, luminosité ambiante.

Chaque graphique est configuré avec des bornes adaptées au paramètre affiché. La température utilise une échelle de 0 à 40°C avec une couleur orange évoquant la chaleur.

L'humidité s'affiche de 0 à 100% en bleu, rappelant l'eau. La luminosité utilise une teinte jaune solaire.

Les données sont récupérées via le provider `readingsHistoryProvider` qui gère automatiquement le cache et le rafraîchissement. Pendant le chargement, un effet de shimmer anime les graphiques pour indiquer l'activité. En cas d'erreur, un message explicite s'affiche avec possibilité de réessayer.

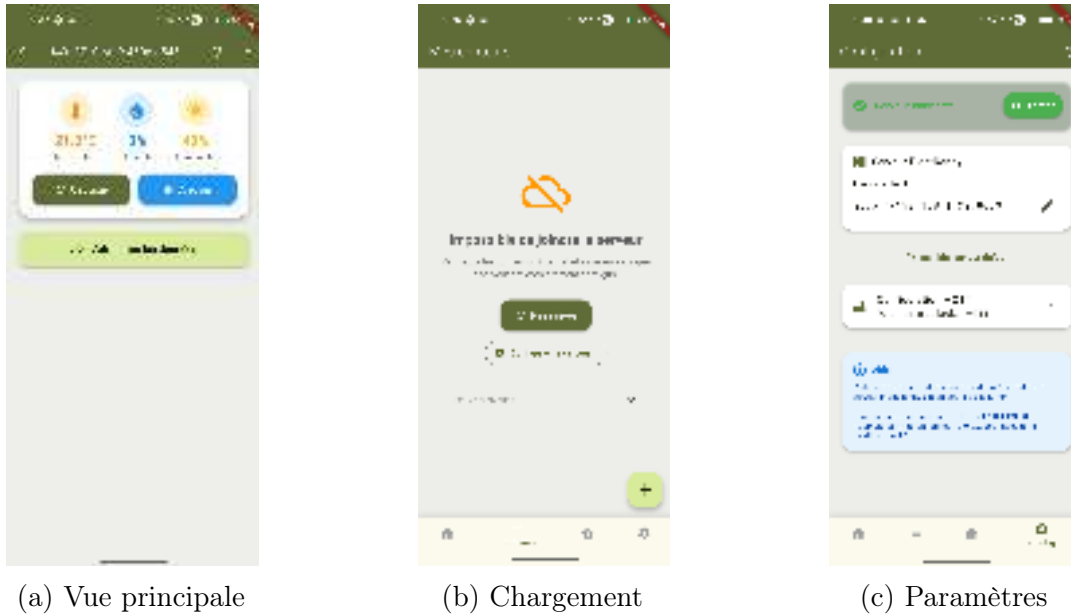


FIGURE 5.1 – Différentes vues de l'interface capteur

5.1.2 Indicateurs Visuels

Au-delà des graphiques, l'application emploie des indicateurs visuels instantanément compréhensibles. L'indicateur d'humidité circulaire combine un arc de progression indiquant le niveau actuel, une icône de goutte d'eau centrée, et une valeur numérique en pourcentage.

La couleur de l'indicateur varie dynamiquement selon le niveau d'humidité, appliquant un code couleur intuitif. Le rouge signale une humidité critique inférieure à 30%, nécessitant un arrosage urgent. L'orange indique une humidité moyenne entre 30% et 50%, suggérant une attention prochaine. Le bleu représente une humidité satisfaisante au-dessus de 50%.

5.2 Contrôle de la LED

5.2.1 Interface de Contrôle

Les capteurs ESP32 sont équipés d'une LED intégrée utilisée, dans le cadre de ce projet, comme un indicateur visuel simulant l'activation de la pompe à eau. En raison de contraintes techniques et de dysfonctionnements de la pompe, l'allumage de la LED permet de signaler que la commande d'arrosage a bien été reçue et exécutée par le capteur.

L'interface de contrôle permet de déclencher cette LED à distance depuis l'application mobile pour une durée définie, correspondant au temps théorique d'écoulement de l'eau programmable soit dans les paramètres du capteurs soit dans l'onglet d'arrosage.

L'interface présente une action principale permettant d'activer la LED pour une durée donnée. Lors du déclenchement, la LED s'allume pendant x secondes, simulant ainsi la mise en marche de la pompe à eau. Une fois cette durée écoulée, la LED s'éteint automatiquement.

Chaque action déclenche l'envoi d'une commande au serveur via le **CommandsRepository**. La commande est ensuite transmise au capteur concerné par l'intermédiaire du broker MQTT. L'interface affiche immédiatement un retour visuel confirmant l'envoi de la commande, sans attendre la fin de son exécution.

Ce mécanisme permet de valider le bon fonctionnement de la chaîne de communication entre l'application, le serveur et le capteur, tout en respectant les contraintes matérielles du projet.

5.3 Interface de Statistiques

5.3.1 Vue d'Ensemble

La page de statistiques offre une vision globale de l'écosystème de capteurs de l'utilisateur, avec des métriques agrégées et des historiques d'activité.

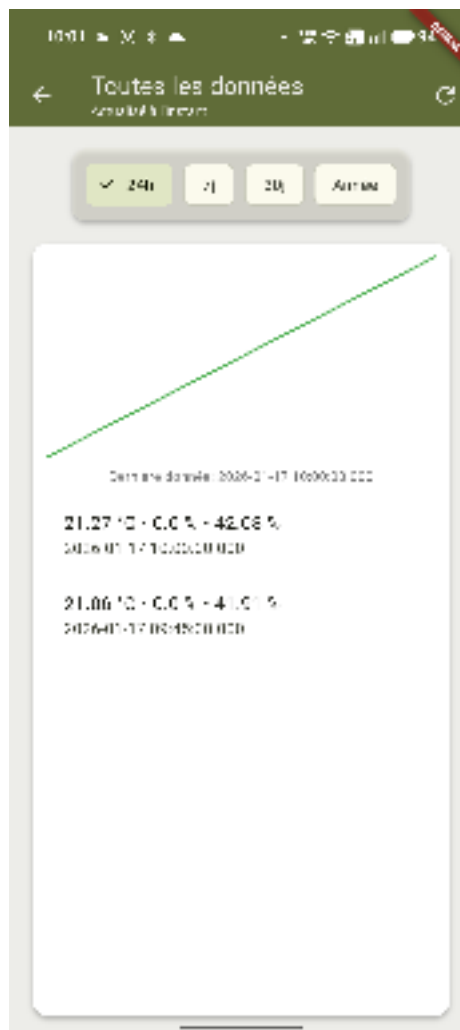


FIGURE 5.2 – Cycle de vie d'une commande

5.4 Contrôle de l'Arrosage (Pompe)

5.4.1 Page d'Arrosage

La page d'arrosage centralise le contrôle des pompes connectées aux capteurs. Elle liste tous les appareils avec leur niveau d'humidité actuel et permet de déclencher l'arrosage individuellement ou globalement.

Chaque capteur est représenté par une carte affichant son nom, son niveau d'humidité actuel avec l'indicateur coloré, et un bouton d'arrosage. L'en-tête de la page comporte un bouton d'action globale permettant d'arroser simultanément toutes les plantes en un seul geste.

5.4.2 Déclenchement de l'Arrosage

Lorsque l'utilisateur déclenche un arrosage, la commande est envoyée au `CommandsRepository` avec l'identifiant du capteur et la durée d'arrosage en millisecondes. Le repository transmet la commande à l'API REST, qui la publie sur le topic MQTT approprié pour le capteur cible.

Un feedback visuel immédiat (snackbar) confirme l'envoi de la commande. L'interface suit ensuite l'état de la commande via le provider de commandes en attente, affichant son statut jusqu'à confirmation d'exécution par le capteur.

En cas d'erreur (capteur hors ligne, timeout), un message d'erreur explicite s'affiche avec la possibilité de réessayer.

5.4.3 Configuration de la Durée

Un dialogue de configuration permet de personnaliser la durée d'arrosage avant envoi. Un slider couvre la plage de 1 à 10 secondes, avec un affichage en temps réel de la durée sélectionnée. Cette flexibilité permet d'adapter l'arrosage aux besoins spécifiques de chaque plante : un arrosage léger de quelques secondes pour les succulentes, ou plus prolongé pour les plantes tropicales.

Les boutons Annuler et Arroser permettent respectivement de fermer le dialogue sans action ou de confirmer l'envoi de la commande avec la durée configurée.

5.5 Provisionnement Bluetooth

5.5.1 Principe du Provisionnement

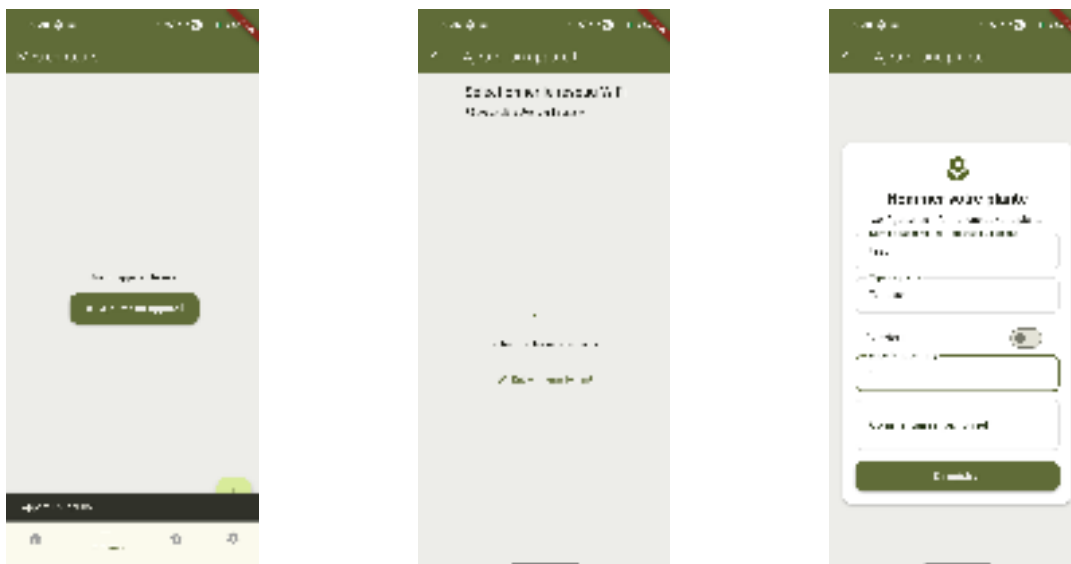
Le provisionnement Bluetooth permet de configurer un nouveau capteur ESP32 sans avoir à le connecter physiquement à un ordinateur. Cette fonctionnalité est essentielle pour une expérience utilisateur fluide : l'utilisateur peut ajouter un capteur directement depuis son smartphone.

Le processus repose sur le protocole **Bluetooth Low Energy (BLE)** et la bibliothèque `flutter_blue_plus`. Le capteur ESP32, non encore configuré, démarre en mode BLE et diffuse son identité sous un nom filtrable (par exemple "PlantNanny-XXXX").

5.5.2 Découverte des Capteurs

Le service Bluetooth de l'application effectue un scan des appareils BLE environnants pendant une durée limitée (typiquement 10 secondes). Seuls les appareils dont le nom correspond au filtre configuré sont retenus, éliminant les autres appareils Bluetooth à proximité.

La liste des capteurs découverts est présentée à l'utilisateur avec leurs identifiants, permettant de sélectionner celui à configurer.



(a) Interface d'ajout de plantes

(b) Recherche wifi

(c) Capteur enregistré

FIGURE 5.3 – Différentes vues de l'appareillage avec l'ESP32

5.5.3 Envoi des Identifiants WiFi

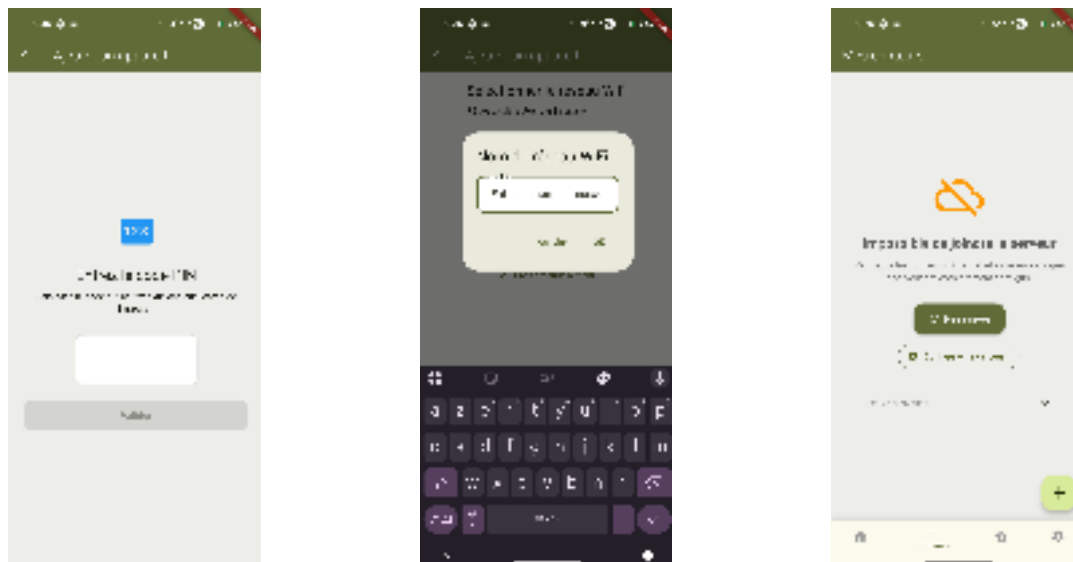
Une fois le capteur sélectionné, l'application établit une connexion BLE et découvre les services exposés par le capteur. Le service de configuration WiFi, identifié par un UUID spécifique, expose une caractéristique permettant d'écrire les identifiants réseau.

L'application encode le SSID et le mot de passe WiFi au format JSON et les transmet via cette caractéristique BLE. Le capteur reçoit ces informations, tente de se connecter au réseau WiFi, et confirme le succès ou l'échec.

La connexion BLE est ensuite fermée proprement, et le capteur redémarre en mode normal, prêt à communiquer avec le serveur via WiFi.

5.5.4 Sécurité du Provisionnement

Le transfert des identifiants WiFi via BLE présente des considérations de sécurité. La portée limitée du BLE (quelques mètres) réduit le risque d'interception. De plus, le mode provisionnement n'est actif que pendant une fenêtre temporelle limitée après le démarrage du capteur, minimisant l'exposition.



(a) Connexion bluetooth

(b) Paramétrage du wifi

(c) Interface sans wifi

FIGURE 5.4 – Différentes vues de l'appareillage avec l'ESP32

5.6 Persistance des Données

5.6.1 Architecture de Stockage

L'application utilise une architecture de stockage hybride combinant plusieurs niveaux de persistance pour optimiser performances et fiabilité.

Les données temps réel (dernières mesures, état des commandes) transitent par l'API REST et sont mises en cache localement via les providers Riverpod. Ce cache en mémoire permet un affichage instantané lors de la navigation entre écrans.

L'historique des mesures est stocké dans la base de données **PostgreSQL** du serveur, accessible via les endpoints d'historique et d'agrégation de l'API.

5.6.2 Mode Hors Ligne

Bien que l'application nécessite une connexion pour les fonctionnalités de contrôle, les dernières données consultées restent accessibles hors ligne grâce au cache local. L'utilisateur peut ainsi consulter l'état de ses plantes même sans connectivité, avec un indicateur signalant l'ancienneté des données affichées.

Chapitre 6

Défis Rencontrés et Solutions Adoptées

Gestion de l'État Asynchrone La gestion des états asynchrones multiples constitue l'un des défis majeurs du développement d'applications mobiles modernes. PlantNanny doit orchestrer simultanément plusieurs sources de données asynchrones : les appels à l'API REST, les communications Bluetooth avec les capteurs, et l'authentification Firebase.

Cette complexité engendre plusieurs risques. Les **race conditions** surviennent lorsque plusieurs requêtes concurrentes tentent de modifier le même état, aboutissant à des résultats imprévisibles. Les **fuites de mémoire** apparaissent quand des subscriptions à des flux de données ne sont pas correctement annulées lors de la destruction des widgets. Les **incohérences d'état** se manifestent lorsque l'interface affiche des données obsolètes ne reflétant plus la réalité côté serveur.

L'adoption de **Riverpod** avec le pattern **AsyncNotifier** résout ces problèmes. Ce pattern encapsule la logique asynchrone dans des contrôleurs dédiés qui gèrent automatiquement leur cycle de vie.

Le mécanisme `ref.onDispose` permet d'enregistrer des callbacks de nettoyage exécutés lors de la destruction du provider. Les timers de rafraîchissement périodique, les subscriptions aux streams, et les connexions réseau sont ainsi libérés proprement, éliminant les fuites de mémoire.

Aussi, le rafraîchissement automatique est implémenté via un timer périodique qui déclenche la mise à jour des données à intervalles réguliers (par exemple toutes les 30 secondes). L'utilisateur voit ainsi des données fraîches sans action manuelle, tout en conservant la possibilité de forcer un rafraîchissement.

La gestion des dépendances entre providers assure la cohérence car lorsqu'un provider dont dépend le contrôleur est modifié (par exemple le token d'authentification), le contrôleur est automatiquement reconstruit avec les nouvelles valeurs.

Communication Bluetooth BLE Le provisionnement du Wi-Fi via Bluetooth Low Energy (BLE) s'est révélé particulièrement complexe à implémenter de manière fiable. Plusieurs difficultés techniques se sont cumulées au cours du développement.

Tout d'abord, les **permissions diffèrent sensiblement entre Android et iOS**. Sur Android, l'accès au scan BLE nécessite non seulement les permissions Bluetooth, mais également celles liées à la localisation. À l'inverse, iOS impose la déclaration explicite des usages du Bluetooth dans le fichier *Info.plist*. La gestion des permissions doit donc être adaptée dynamiquement en fonction de la plateforme cible.

Par ailleurs, des **timeouts de connexion fréquents** ont été observés, en particulier

sur certains appareils Android dont la pile Bluetooth peut se révéler instable. Une procédure fonctionnelle sur un modèle de téléphone pouvait échouer systématiquement sur un autre. Un problème récurrent rencontré concernait l'appairage : dès que le capteur détectait un point d'accès Wi-Fi, le processus de provisionnement était interrompu prématurément, cette détection étant interprétée à tort comme une réussite, empêchant ainsi la finalisation complète de la connexion Wi-Fi.

Afin de pallier ces difficultés, une **logique de réessai (retry)** a été mise en place pour gérer de manière transparente les échecs transitoires. Chaque opération est tentée jusqu'à trois fois, avec un délai croissant entre chaque tentative. Seul un échec définitif, après épuisement de l'ensemble des tentatives, est remonté à l'utilisateur.

Des **timeouts explicites** ont également été définis pour chaque étape critique du processus : connexion au périphérique, découverte des services BLE et écriture des caractéristiques. Ces délais permettent d'éviter que l'application ne reste bloquée indéfiniment sur une opération non répondante.

Enfin, en cas d'échec, la connexion BLE est systématiquement fermée avant toute nouvelle tentative. Cette approche évite l'accumulation de connexions résiduelles, susceptibles de saturer les ressources Bluetooth du terminal et de dégrader la fiabilité globale du système.

Performance des Graphiques L'affichage de graphiques représentant l'historique des mesures sur plusieurs jours implique potentiellement des centaines voire des milliers de points de données. Le rendu naïf de tous ces points provoque des ralentissements perceptibles, particulièrement sur les appareils d'entrée de gamme.

Plusieurs techniques d'optimisation sont combinées pour garantir des performances fluides.

Le **downsampling** réduit le nombre de points affichés à un maximum adapté à la résolution de l'écran (typiquement 200 points). Les points intermédiaires sont moyennés par intervalles, préservant la forme générale de la courbe tout en réduisant drastiquement le nombre de primitives à dessiner.

L'utilisation de **RepaintBoundary** isole le graphique dans sa propre couche de composition. Les repaints du graphique n'affectent plus le reste de l'interface, et inversement. Cette isolation élimine les repaints inutiles lors d'animations ailleurs dans l'écran.

Un **cache de Path** stocke le tracé calculé du graphique. Tant que les données et la taille d'affichage ne changent pas, le même Path est réutilisé sans recalcul. Seul le redessin du Path en cache est effectué, opération très rapide.

Ces optimisations combinées permettent d'afficher des graphiques complexes à 60 FPS même sur des appareils modestes.

Problème	Impact	Solution
États asynchrones	Incohérences UI, fuites mémoire	Riverpod + AsyncNotifier avec cleanup automatique
Bluetooth BLE	Échecs fréquents, incompatibilités	Couche d'abstraction avec retry logic et timeouts
Synchronisation données	Duplications, données obsolètes	Cache avec politique de fraîcheur configurable
Performance graphiques	Lenteurs, saccades	Downsampling, Repaint-Boundary, cache de Path
Testabilité	Tests difficiles voire impossibles	Interfaces abstraites + injection de dépendances
Erreurs utilisateur	Messages cryptiques	Traduction contextuelle avec actions de retry
Multi-environnements	Erreurs de configuration	Flavors Flutter avec configuration compile-time

TABLE 6.1 – Tableau récapitulatif des différents obstacles rencontrés durant le développement

Chapitre 7

Conclusion

Le projet **PlantNanny** avait pour objectif de concevoir une application mobile capable de superviser et piloter un ensemble de capteurs IoT dédiés au suivi et à l'arrosage des plantes. À l'issue de ce projet, les objectifs fixés ont été atteints.

De ce fait, l'application permet de consulter en temps réel les données issues des capteurs (température, humidité du sol, luminosité), de visualiser leur évolution grâce à des graphiques, et de déclencher des actions à distance via une architecture reposant sur une API REST et un broker MQTT. Le provisionnement des capteurs par Bluetooth Low Energy, bien que complexe à implémenter, améliore considérablement l'expérience utilisateur en simplifiant l'ajout de nouveaux dispositifs.

L'architecture adoptée, basée sur les principes de l'architecture propre, a démontré sa pertinence en facilitant la maintenance, l'évolution du projet et la mise en place de tests. La séparation claire entre présentation, logique métier et accès aux données a permis de développer une application robuste et évolutive.

Au-delà des aspects techniques, ce projet a été une expérience formatrice, tant sur le plan du développement mobile avec Flutter que sur l'intégration de systèmes IoT et le travail en équipe. Les compétences acquises constituent une base solide pour de futurs projets, et les nombreuses pistes d'évolution identifiées laissent entrevoir un fort potentiel d'amélioration de l'application.